Text2SQL: Ablation Study & Feature Engineering

BIRD + Ambrosia Benchmark Results

Содержание

Цель и стартовая точка 3
Методология: Ablation v2 (leave-one-in) 5
Результаты фич: группы A и B 7
Результаты фич: группы C и D 9
Почему FEAT_2 дала самый большой прирост (+9.09%) 11
Структурная проблема: FEAT_32/33 в isolation дают −8% 13
Прогрессия гибридных экспериментов 15
Архитектурный инсайт: два несовместимых режима 17
Оптимальная комбинация фич 19
Валидация на Large датасетах 21
Анализ оставшихся ошибок BIRD (10 из 22) 23
Итоги и выводы 26
Спасибо за внимание! 29

Цель и стартовая точка

Построить Text2SQL пайплайн, решающий две задачи одновременно: **BIRD** — генерация корректного SQL по вопросу на естественном языке, и **Ambrosia** — детекция неоднозначных вопросов.

Метрика	Baseline	Цель	Достигнуто
BIRD	36.36%	≥ 50%	**54.55%**
Ambrosia	54.17%	≥ 70%	**91–96%**

Baseline = все флаги отключены (чистый пайплайн: вопрос → LLM → SQL).

Методология: Ablation v2 (leave-one-in)

Принцип

Baseline = все флаги выключены (чистый пайплайн, 36.36% BIRD / 54.17% AMB). Каждая фича тестируется **в isolation** — только она одна включена, всё остальное выключено.

Что измеряем: Δ BIRD и Δ AMB относительно этого чистого baseline.

v1 vs v2

В v1 фиксировался base stack (FEAT_5+12+18+19) — результаты оказались недостоверными: без FEAT_12 ambiguity-фильтр помечал 60–80% вопросов как ambiguous, без FEAT_18/19 rate-limit cascade убивал прогон. В **v2** каждая фича тестируется от нуля, в полной изоляции.

Датасеты

- **BIRD small:** 22 вопроса, 7 БД → 1 вопрос = 4.55% (высокий шум)
- **Ambrosia small:** 24 вопроса, 9 БД

Почему не fold-out

При ≤ 24 вопросах leave-one-out даёт оценку с высокой дисперсией. Leave-one-in честнее: видно, добавляет ли фича что-то поверх стабильного baseline.

Результаты фич: группы A и B

Группа A: Обогащение схемы

Фича	Что делает	BIRD Δ	AMB Δ
FEAT_2	Column statistics: min/max/top-K, кардинальность	**+9.09%**	0%
FEAT_3	Regex-типизация TEXT-колонок	+4.55%	0%
FEAT_4	pg_stat row-count таблиц	+4.55%	+4.16%
FEAT_35	TSV-формат схемы вместо JSON	+4.55%	+4.16%
FEAT_1	FK/PK в промпте	**−9.09%**	+4.16%
FEAT_6	Heavy schema (sample rows)	0%	0%

Группа B: Качество генерации SQL

Фича	Что делает	BIRD Δ	AMB Δ
FEAT_12	Optimistic fallback при ambiguous-флаге	+4.55%	**+8.33%**
FEAT_29	Классификация сложности + разные промпты	+4.55%	0%
FEAT_30	CoT decomposition (sub-questions)	+4.55%	0%
FEAT_20	LLM-as-judge в retry	−4.55%	−4.17%
FEAT_17	Retry loop (MAX_RETRIES=7)	−4.55%	0%
FEAT_23	NL-описания таблиц	−4.55%	0%

Результаты фич: группы C и D

Группа C: AmbiSQL Pipeline (детекция неоднозначности)

Фича	Что делает	BIRD Δ	AMB Δ
FEAT_32	Taxonomy-детекция A1–A6	0%	−8.34%*
FEAT_33	Генерация clarification-вопросов	−4.55%	−8.34%*
FEAT_34	Auto-resolve: LLM выбирает ответ	0%	+8.33%

**В isolation плохи из-за ошибок фреймирования промпта и JSON parse errors. В комбо с исправленным FEAT_32 → AMB 91–96%.*

Группа D: Инфраструктура

Фича	Эффект
FEAT_8	PG rollback — предотвращает cascade
FEAT_17	Retry loop — усиливает FEAT_27
FEAT_18	Exponential backoff на 429
FEAT_19	asyncio.sleep — страховка FEAT_18
FEAT_27	True self-correction + FEAT_17

Почему FEAT_2 дала самый большой прирост (+9.09%)

FEAT_2 = column statistics (min/max/top-K, cardinality)

```
Без FEAT_2: LLM видит только "date" типа TEXT
С FEAT_2:   min="199801", max="201212",
            sample=["200301","199901"]
```

Три класса ошибок, которые FEAT_2 устраняет

1. **Неправильный формат дат.** `yearmonth.Date = "YYYYMM"` → без статистики LLM генерирует `WHERE date = '2012-01'` → пустой результат. Со статистикой: `WHERE date = '201201'`.
2. **Лишние CAST.** Зная min=0, max=1000 — LLM не оборачивает в `TO_DATE()` числовую колонку.
3. **Подбор порогов.** Зная диапазон `AvgScrMath` ∈ [300, 800] — LLM правильно выбирает `WHERE AvgScrMath > 560`.

Почему FEAT_3/4 дали меньше

- **FEAT_3** (regex-типизация) применима только к TEXT-колонкам, которых в BIRD-схемах меньшинство
- **FEAT_4** (row count) полезен для LIMIT-решений, но таких вопросов в small всего 1–2
- Они частично чинят **те же вопросы**, что и FEAT_2 — не-аддитивность сигналов

Итог: FEAT_2 устраняет системные ошибки формата. FEAT_3/4 полезны, но перекрываются — реальный комбо-прирост 12% вместо арифметических 18%.

Структурная проблема: FEAT_32/33 в isolation дают −8%

Проблема 1: фреймирование промпта

```
Было: "По умолчанию считай запрос
ОДНОЗНАЧНЫМ. Неоднозначность — исключение"
→ 0/9 gold-ambiguous вопросов детектировалось
```

```
Стало: "Это задача сопоставления
синтаксического паттерна,
не философское суждение"
→ 21/24 gold-ambiguous вопросов детектировалось
```

Проблема 2: JSON parse errors

LLM добавлял текст после `}` или вставлял raw newlines внутри строк → 8/24 вопросов падали с `JSONDecodeError` → FEAT_12 fallback → taxonomy игнорировалась.

Фикс:

```
# Было (сломано):
text.replace("\n", "\\n")
# структурные переводы строк → невалидный JSON

# Стало:
re.sub(r"[\x00-\x1f]", " ", text)
# пробел валиден в любом месте JSON
```

Прогрессия гибридных экспериментов

Эксп.	Фичи	BIRD	AMB	Ключевое изменение
Baseline	—	36.36%	54.17%	Стартовая точка
Hybrid 1	FEAT_2+12+32+33+35	40.91%	70.83%	Taxonomy: философия → паттерны
Hybrid 2	+FEAT_3+4+8+17+18+19+27+29+30	50.00%	66.67%	BIRD-фичи добавлены; JSON errors не пофикшены
Hybrid 3	те же	50.00%	87.50%	JSON парсер + counterexamples для FP/FN
Hybrid 4	те же	**54.55%**	**95.83%**	Alias check rule в SQL_PROMPT
Hybrid 5	те же	54.55%	91.67%	TO_DATE retry-rule

Почему AMB упал 70.83% → 66.67% при добавлении BIRD-фич (H2)? JSON parse errors: FEAT_32 падала на 8/24 вопросах → FEAT_12 fallback → taxonomy игнорировалась.

Почему Hybrid 4 дал +4.55% BIRD? Alias check rule устранил Q685 (case sensitivity) и Q17 (необъявленный alias в FROM).

Архитектурный инсайт: два несовместимых режима

FEAT_34 (auto-resolve) создаёт жёсткий потолок AMB

```
AmbiSQL pipeline:
  Stage 1: detect_ambiguity_by_taxonomy()
  Stage 2a: generate_clarification_question()
  Stage 2b: auto_resolve_ambiguity() →
            LLM выбирает → переписывает → SQL
```

При FEAT_34=True: **любой** вопрос в итоге генерирует SQL. `{"status": "ambiguous"}` **никогда не возвращается**. AMB accuracy = % угаданных вариантов ≤ 62.5%.

Режим	FEAT_34	BIRD	AMB
Detect-only	✗	54.55%	91–96%
Auto-resolve	✓	45%	≤ 62.5%

Вывод: FEAT_34 полезна в production (нет живого пользователя для уточнения), но **несовместима с бенчмарк-метрикой AMB**.

Оптимальная комбинация фич

```
FEAT_2 + FEAT_3 + FEAT_4 + FEAT_8 + FEAT_12 + FEAT_17 + FEAT_18 + FEAT_19 + FEAT_27 + FEAT_29 + FEAT_30 + FEAT_32 + FEAT_33 + FEAT_35
```

Итог: BIRD 54.55% / AMB 91–96%

Роль каждой группы

Группа	Фичи	Вклад
Контекст для LLM	2, 3, 4, 35	Статистика + типы + компактный формат
Качество SQL	29, 30, 27, 17	CoT + self-correction + retry loop
Детекция AMB	32, 33	Taxonomy + clarification (без auto-resolve)
Защита от FP	12	Fallback при сбое taxonomy
Инфраструктура	8, 18, 19	PG rollback + rate-limit защита

Что исключено

Фича	Причина
FEAT_1	−9.09% BIRD: перегружает промпт
FEAT_5	−4.55%/−4.17%: хуже baseline; FEAT_35 заменяет
FEAT_20	−4.55% BIRD: LLM-судья накапливает ложные отказы
FEAT_34	AMB потолок 62.5%: несовместим с целью 70%+

Валидация на Large датасетах

BIRD Large: 41.49% (100/241)

Сложность	Точность	Кол-во
Simple	57.14%	91
Moderate	39.33%	89
Challenging	21.31%	61

Контекст: large добавляет БД `superhero` (новый домен) и 25% `challenging` вопросов vs 9% в small → прямое сравнение с 36.36% некорректно.

Ambrosia Large: 57.52%

Без FEAT_34: +3.35pp vs baseline. Для максимального AMB на large — нужен FEAT_32+33 в режиме detect-only.

Датасет	Small	Large	Статус
BIRD	22 вопр.	241 вопр.	✅
Ambrosia	24 вопр.	240 вопр.	✅ (52/55 БД)

Анализ оставшихся ошибок BIRD (10 из 22)

Вопрос	Тип ошибки	Причина	Исправимо?
Q417, Q1533	Broken gold	`NULLIF(NULLIF,0)(...)` — невалидный SQL в бенчмарке	✗
Q1472	Семантика	LAM = страны из gasstations, gold использует `customers.segment`	✗
Q1529	Семантика	`amount*price` вместо `SUM(Price)`	✗
Q1388	Семантика	MAX вместо SUM для «highest income»	✗
Q13	Семантика	Лишний фильтр сужает выборку	✗
Q17	Семантика+SQL	Нет `RANK() OVER (...)`	⚠ Частично
Q180	SQL	`TO_DATE(date)` на DATE-колонку + сложный 4-table JOIN	⚠
Q345	SQL	Лишние колонки (gold ожидает только `status`)	⚠
Q1365	SQL	Нет `ORDER BY ... NULLS FIRST`	⚠

Эффективный потолок без смены модели: 59% (13/22 при устранении Q17/Q180/Q345). 5 из 10 ошибок — семантические или broken gold → не решаются промпт-инжинирингом.

Итоги и выводы

Что работает

1. **Статистика данных важнее структуры.** FEAT_2 (+9.09%) обгоняет FEAT_1 FK-связи (−9.09%). LLM эффективнее использует min/max/top-K, чем граф зависимостей.
2. **Не-аддитивность сигналов.** FEAT_2+3+4 ≠ +18.19%. В комбинации они чинят те же вопросы → реальный прирост 12%.
3. **Фреймирование промпта критично.** FEAT_32 в isolation: −8% AMB → после переписывания как паттерн-матчинг: +37% AMB в hybrid.
4. **Инфраструктурные фичи — обязательный base-stack.** Без FEAT_18/19 rate-limit cascade убивает прогон.

Финальные числа

Метрика	Baseline	Итог	Δ
BIRD	36.36%	**54.55%**	**+18.19pp**
AMB	54.17%	**91–96%**	**+37–42pp**

- **BIRD:** +50% относительный прирост
- **AMB:** +69–78% относительный прирост

5. **AMB и auto-resolve несовместимы.**
   FEAT_34 хороша в production, но блокирует AMB-метрику на 62.5%.

Спасибо за внимание!